ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

Assignment 2: Fine-tuning Deepseek OCR với Dataset Tiếng Việt

23CNTThuc1 - Nhập môn xử lý ngôn ngữ tự nhiên

Giảng viên hướng dẫn:

Nguyễn Hồng Bửu Long

Đinh Điền

Lương An Vinh

Sinh viên thực hiện:

Trần Phụng Đình (23127527)

Ngày 4 tháng 1 năm 2026

Mục lục

1 Thông tin sinh viên **1**

2 Tổng quan đồ án **1**

2.1 Tổng quan về bài toán OCR . . . 1

2.2 Những khó khăn & thử thách trong bài toán OCR Tiếng Việt . . . 1

3 Tổng quan mô hình DeepSeek-OCR **2**

3.1 Kiến trúc mô hình . . . 2

3.2 Input và Output . . . 3

3.3 Ứng dụng . . . 3

4 Phương pháp **3**

4.1 Thu thập dữ liệu . . . 3

a) Mô tả nguồn dữ liệu . . . 3

b) Chuẩn bị dữ liệu (Data Preparation) . . . 4

4.2 Quy trình Finetune . . . 4

a) Cài đặt môi trường (Environment Setup) . . . 4

b) Chi tiết cấu hình Fine-tuning . . . 4

c) Phân tích quá trình hội tụ (Training Convergence Analysis) . . . 7

4.3 Quy trình Suy luận và Hậu xử lí . . . 9

5 Kết quả và đánh giá **9**

5.1 Chiến lược dữ liệu (Data Strategy) . . . 9

a) Lấy mẫu phân tầng (Stratified Sampling) . . . 9

b) Thiết kế kịch bản thực nghiệm . . . 9

5.2 Chỉ số đánh giá . . . 10

5.3 Kết quả thực nghiệm . . . 10

a) Kết quả định lượng (Quantitative Results) . . . 10

b) Phân tích lỗi định tính (Qualitative Analysis) . . . 11

6 Kết luận **12**

6.1 Những phát hiện chính (Key Findings) . . . 12

6.2 Kết luận . . . 12

7 Tài liệu **13**

8 Source code **13**

9 Acknowledgement **13**

1 Thông tin sinh viên

Họ tên	MSSV	Email	Lớp
Trần Phụng Đình	23127527	tpdinh23@clc.fitus.edu.vn	23CNTThuc1

2 Tổng quan đồ án

2.1 Tổng quan về bài toán OCR

OCR, viết tắt là Optical Character Recognition (Nhận dạng ký tự quang học), là tiến trình chuyển đổi hình ảnh chứa văn bản (tài liệu scan, ảnh chụp cảnh quan, biển số xe, ...) thành dữ liệu văn bản dạng máy có thể đọc và chỉnh sửa được (machine-readable text).
Các hệ thống OCR hiện đại thường không còn dựa vào phương pháp so khớp mẫu (*template matching*) truyền thống, mà chuyển sang sử dụng Deep Learning (Học sâu) với quy trình gồm hai giai đoạn chính:

- **Text Detection (Phát hiện vùng chữ)**: Xác định vị trí của các vùng văn bản trong ảnh, thường được biểu diễn dưới dạng khung bao (*bounding box*). Các mô hình tiêu biểu bao gồm: CRAFT, DBNet, EAST.
- **Text Recognition (Nhận dạng nội dung)**: Chuyển đổi các điểm ảnh trong vùng văn bản đã được phát hiện thành chuỗi ký tự có nghĩa. Các kiến trúc phổ biến gồm: CRNN (CNN + RNN + CTC Loss) và các mô hình dựa trên Transformer.

OCR được ứng dụng rộng rãi trong nhiều lĩnh vực như số hóa tài liệu, trích xuất thông tin định danh điện tử (eKYC), hỗ trợ người khiếm thị và dịch thuật thời gian thực.

2.2 Những khó khăn & thử thách trong bài toán OCR Tiếng Việt

Mặc dù các mô hình OCR cho tiếng Anh (Latin) đã đạt độ chính xác rất cao (>99%), việc áp dụng trực tiếp cho tiếng Việt gặp phải sự sụt giảm hiệu suất đáng kể do các đặc thù ngôn ngữ sau:

1. **Hệ thống dấu thanh và ký tự phụ (Diacritics)**

   Đây là thách thức lớn nhất trong nhận dạng ký tự tiếng Việt. Ngôn ngữ này sở hữu hệ thống nguyên âm phức tạp với nhiều dấu phụ như *ă, â, ê, ô, ơ, ư* cùng năm dấu thanh gồm: sắc, huyền, hỏi, ngã và nặng.

   Vấn đề: Các dấu thanh và dấu phụ thường chiếm diện tích pixel rất nhỏ trong ảnh.

   Hệ quả: Trong điều kiện ảnh bị mờ (*blur*), nhiễu (*noise*) hoặc có độ phân giải thấp, mô hình OCR dễ xảy ra nhầm lẫn, chẳng hạn như nhầm dấu sắc thành dấu huyền, dấu nặng thành nhiễu ảnh hoặc bỏ sót hoàn toàn dấu mũ (^).

2. **Cấu trúc ký tự chồng chéo (Stacked Diacritics)**

   Tiếng Việt cho phép kết hợp đồng thời dấu nguyên âm và dấu thanh trên cùng một ký tự, ví dụ như *ấ, ệ, ể, ễ, ồ, ở.*

   Vấn đề: Việc chồng nhiều dấu lên một ký tự làm cho chiều cao của dòng chữ (*line height*) thay đổi không đồng nhất.

Hệ quả: Điều này gây khó khăn cho các thuật toán cắt dòng văn bản và quá trình chuẩn hóa kích thước ảnh đầu vào của mô hình nhận dạng.

3. **Sự nhập nhằng về hình thái (Visual Ambiguity)**

   Một số ký tự tiếng Việt có hình thái rất tương đồng, đặc biệt trong chữ viết tay hoặc khi sử dụng các kiểu font không chuẩn.

   - Ký tự *d* và *đ* chỉ khác nhau bởi một nét gạch ngang nhỏ.
   - Dấu hỏi (') và dấu ngã (~) thường bị nhầm lẫn trong quá trình nhận dạng chữ viết tay.

4. **Hạn chế về dữ liệu huấn luyện (Dataset Limitation)**

   So với các ngôn ngữ phổ biến như tiếng Anh hoặc tiếng Trung, lượng dữ liệu gán nhãn chất lượng cao (*labeled data*) cho tiếng Việt, đặc biệt là dữ liệu chữ viết tay (*handwritten text*), còn khá hạn chế.

   Hệ quả: Điều này khiến mô hình OCR khó đạt được khả năng tổng quát hóa (*generalization*) tốt khi áp dụng cho dữ liệu thực tế đa dạng.

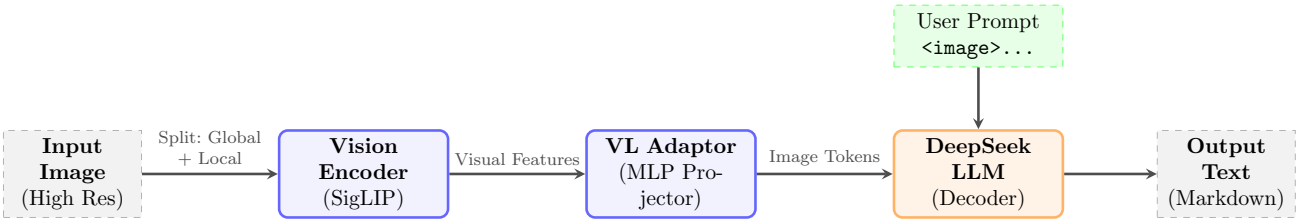
3 Tổng quan mô hình DeepSeek-OCR

3.1 Kiến trúc mô hình

DeepSeek-OCR được xây dựng dựa trên nền tảng của kiến trúc **DeepSeek-VL-V2**, một mô hình đa phương thức (Vision-Language Model) tiên tiến. Kiến trúc bao gồm ba thành phần cốt lõi:

1. **Vision Encoder (SigLIP):** Bộ mã hóa hình ảnh hiệu năng cao, chịu trách nhiệm trích xuất đặc trưng thị giác từ ảnh đầu vào.
2. **Vision-Language Adaptor (MLP Projector):** Cầu nối giúp chuyển đổi các đặc trưng hình ảnh sang không gian embedding của ngôn ngữ, cho phép mô hình "hiểu" được hình ảnh như các token văn bản.
3. **Large Language Model (DeepSeek-LLM):** Phần não bộ xử lý suy luận, tiếp nhận chuỗi token hỗn hợp (ảnh + văn bản) để sinh ra văn bản kết quả.

Điểm đặc biệt giúp DeepSeek-OCR vượt trội trong việc nhận dạng văn bản là cơ chế **Dynamic High-Resolution** (Xử lý độ phân giải động). Ảnh đầu vào không bị nén cứng nhắc mà được chia thành các mảnh nhỏ (local crops) kết hợp với một cái nhìn toàn cảnh (global view), giúp mô hình đọc được cả chi tiết nhỏ nhất (dấu câu, nét chữ).



Hình 1: Sơ đồ luồng dữ liệu trong kiến trúc DeepSeek-OCR

3.2 Input và Output

- **Input:**
  - Ảnh văn bản (hỗ trợ độ phân giải cao, đa tỉ lệ).
  - Câu lệnh (Prompt): Thường có dạng `<image> Analyze this text...` hoặc `<image> Free OCR`.
- **Output:** Chuỗi văn bản thuần túy hoặc định dạng cấu trúc (Markdown, LaTeX, JSON) mô tả chính xác nội dung trong ảnh.

3.3 Ứng dụng

Nhờ khả năng hiểu ngữ nghĩa sâu sắc, mô hình phù hợp cho:

- **Số hóa tài liệu:** Chuyển đổi sách, báo cáo, chữ viết tay sang văn bản điện tử.
- **Trích xuất thông tin:** Đọc hóa đơn, biểu mẫu, bảng biểu phức tạp.
- **Hỗ trợ người khiếm thị:** Mô tả nội dung văn bản trong ảnh chụp đời sống.

4 Phương pháp

4.1 Thu thập dữ liệu

a) Mô tả nguồn dữ liệu

Dataset được sử dụng trong nghiên cứu này là **UIT_HWDB** (University of Information Technology - Handwriting Database). Dưới đây là các thông tin chi tiết:

- **Nguồn dữ liệu:** Dataset được phát triển bởi nhóm nghiên cứu tại Trường Đại học Công nghệ Thông tin, ĐHQG-HCM (UIT). Dữ liệu được công bố kèm theo bài báo khoa học tại hội nghị RIVF 2022.
- **Mô tả ngắn gọn:** UIT-HWDB là một bộ dữ liệu tiếng Việt phục vụ bài toán nhận dạng chữ viết tay không ràng buộc (unconstrained handwriting recognition). Dataset này được tạo ra bằng phương pháp "Transferring Method", chuyển đổi dữ liệu chữ viết tay online (nét bút) sang dạng ảnh offline với các đặc trưng tự nhiên về màu sắc, độ đậm nhạt và nhiễu nền, giúp mô phỏng sát với dữ liệu thực tế. Cấu trúc dữ liệu bao gồm các mức độ: từ đơn (word), dòng (line) và đoạn văn (paragraph).
- **Kích thước dữ liệu:** Tổng cộng dataset bao gồm các phần chính sau:
  - **UIT-HWDB-word:** 110,745 ảnh từ.
  - **UIT-HWDB-line:** 7,273 ảnh dòng.
  - **UIT-HWDB-paragraph:** 1,144 ảnh đoạn văn.

b) Chuẩn bị dữ liệu (Data Preparation)

Để khắc phục nhược điểm về tốc độ I/O khi đọc hàng nghìn tệp ảnh nhỏ rải rác, em đã thực hiện **đóng gói toàn bộ dữ liệu thô sang định dạng nhị phân Apache Arrow** thông qua thư viện `datasets`.

Cấu trúc Dataset: Mỗi mẫu dữ liệu bao gồm 3 trường thông tin (features) chính:

- `image`: Dữ liệu ảnh thô (Image Object).
- `text`: Nhãn văn bản (Ground Truth) trích xuất từ JSON.
- `type`: Phân loại mức độ ảnh (*word, line, paragraph*) giúp đảm bảo tỉ lệ hợp lí giữa các loại ảnh khi lọc subset dữ liệu .

Quy trình xử lý dữ liệu:

1. **Quét dữ liệu:** Hệ thống quét đệ quy thư mục gốc `UIT_HWDB`.
2. **Đối khớp và Làm sạch:** Tại mỗi thư mục con, script tự động tìm tệp `label.json` và đối chiếu với file ảnh. Các mẫu lỗi (thiếu ảnh hoặc thiếu nhãn) bị được bỏ qua.
3. **Phân chia tập dữ liệu:** Dựa trên cấu trúc thư mục đã phân tầng, dữ liệu được chia thành hai tập **Train** và **Test**.
4. **Đóng gói:** Kết quả cuối cùng là đối tượng `DatasetDict` được lưu dưới dạng Arrow và nén thành tệp zip.

Source Notebook chuẩn bị dữ liệu: **Kaggle - [NLP] Prepare Input**

4.2 Quy trình Finetune

a) Cài đặt môi trường (Environment Setup)

Quá trình huấn luyện được thực hiện trên Kaggle với cấu hình phần cứng và phần mềm như sau:

- **Phần cứng:** 2x Tesla T4 GPU (14.7GB VRAM mỗi GPU).
- **Thư viện:**
  - `unsloth`: Tối ưu hóa tốc độ tải model nhanh hơn và giảm tiêu thụ bộ nhớ.
  - `transformers`, `peft`, `trl`: Hệ sinh thái chuẩn của Hugging Face.
- **Base Model:** `unsloth/DeepSeek-OCR` (phiên bản lượng tử hóa 4-bit qua `FastVisionModel`).

b) Chi tiết cấu hình Fine-tuning

Quá trình tinh chỉnh sử dụng phương pháp **QLoRA** (Quantized Low-Rank Adaptation) để phù hợp với tài nguyên hạn chế.

1. Cấu hình Model (QLoRA Configuration): Mô hình được tải ở độ chính xác 4-bit (NF4), sau đó gắn các adapter LoRA vào các lớp tính toán trọng yếu:

- **Rank (r) & Alpha (α):** $r = 16$, $\alpha = 16$ (Scaling factor = 1).
- **Target Modules:** Toàn bộ các lớp chiếu trong khối Attention và Feed-Forward: `[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]`.
- **Bias & Dropout:** Không sử dụng (`Bias="none"`, `Dropout=0`), nhằm tiết kiệm VRAM và cho phép mô hình tận dụng tối đa khả năng học của số lượng tham số giới hạn.

2. Cấu hình xử lý ảnh (Image Processing) Sử dụng `DeepSeekOCRDataCollator` với cơ chế **Dynamic Resolution** (Độ phân giải động) để tối ưu hóa nhận dạng đa tỷ lệ:

- **Chiến lược:** Kết hợp cái nhìn toàn cảnh (Global view) và chi tiết cục bộ (Local crops).
- **Kích thước đầu vào:** Thiết lập Global (768×768) và Local patch (512×512) thay vì kích thước gốc (1024×1024). Đây là bước xử lý cần thiết để thích ứng với ràng buộc phần cứng (hardware constraints), ngăn chặn hiện tượng tràn bộ nhớ (OOM) khi huấn luyện với kỹ thuật gradient accumulation.
- **Giới hạn lưới cắt (Grid Limit):** Tham số `max_num=2` được thiết lập để giới hạn số lượng mảnh cắt cục bộ tối đa (tối đa lưới 2×2 hoặc 1×2), giúp kiểm soát tổng số lượng **visual tokens** sinh ra từ một ảnh. Nếu không giới hạn, các ảnh văn bản có khổ quá dài hoặc rộng sẽ bị chia thành quá nhiều mảnh nhỏ, dẫn đến bùng nổ số lượng token đầu vào vượt quá Context Window của mô hình và VRAM của GPU.
- **Thông số khác:** `Patch size = 16`, `Downsample ratio = 4`.

3. Siêu tham số tối ưu hóa (Hyperparameters) Chi tiết các tham số và chiến lược huấn luyện được trình bày trong Bảng 1.

Bảng 1: Chi tiết các siêu tham số huấn luyện (Training Hyperparameters)

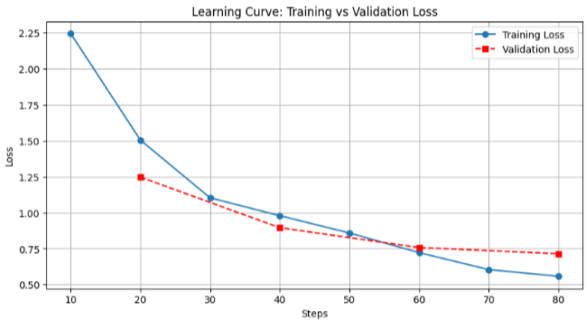
Tham số	**Cấu hình**	**Ý nghĩa**
Epochs	3	Số lần mô hình duyệt qua toàn bộ tập dữ liệu huấn luyện.
Batch Size	1 (per device) $\times$ 16 (gradient accumulation) $\rightarrow$ **Effective Batch Size = 16**	Sử dụng kỹ thuật tích lũy gradient để mô phỏng batch size lớn, giúp hội tụ ổn định trên GPU giới hạn bộ nhớ (T4).
Learning Rate	2×10^{-4} (Linear Scheduler)	Tốc độ học khởi điểm; giảm dần tuyến tính về 0 để tinh chỉnh trọng số tốt hơn ở cuối quá trình.
Weight Decay	0.01	Cơ chế điều hòa (Regularization) giúp trừng phạt các trọng số lớn, giảm thiểu hiện tượng overfitting.
Warmup Steps	50 bước	Tăng dần Learning Rate từ 0 trong 50 bước đầu để ổn định mô hình trước khi học chính thức.
Save/Eval Steps	20	Tần suất thực hiện đánh giá trên tập Validation và lưu checkpoint, giúp theo dõi quá trình hội tụ liên tục, phát hiện sớm thời điểm mô hình đạt hiệu năng tốt nhất (early stopping) hoặc bắt đầu bị overfitting..
Optimizer	`adamw_8bit`	Phiên bản tối ưu hóa AdamW sử dụng 8-bit cho các trạng thái (states), tiết kiệm VRAM đáng kể.
Precision	FP16 (Mixed Precision)	Huấn luyện với độ chính xác hỗn hợp giúp tăng tốc độ tính toán mà không giảm chất lượng đáng kể.
Loss Function	Cross Entropy (`train_on_responses_only=True`)	Hàm mất mát chỉ tính lỗi trên phần văn bản đầu ra (response), bỏ qua phần prompt đầu vào.
Seed	3407	Hạt giống ngẫu nhiên cố định để đảm bảo kết quả thực nghiệm có tính tái lập (reproducible).

Source Notebook finetune và đánh giá model: **Kaggle - [NLP] Deepseek OCR Finetune**

c) Phân tích quá trình hội tụ (Training Convergence Analysis)

	Step	Epoch	Train Loss	Val Loss	LR
0	10	0.36	2.2445	NaN	0.000036
1	20	0.71	1.5047	1.247500	0.000076
2	30	1.04	1.1037	NaN	0.000116
3	40	1.39	0.9796	0.896256	0.000156
4	50	1.75	0.8593	NaN	0.000196
5	60	2.07	0.7229	0.758330	0.000151
6	70	2.43	0.6052	NaN	0.000097
7	80	2.78	0.5586	0.715260	0.000043
8	87	3.00	NaN	NaN	NaN

(a) Training Log (1000 ảnh)

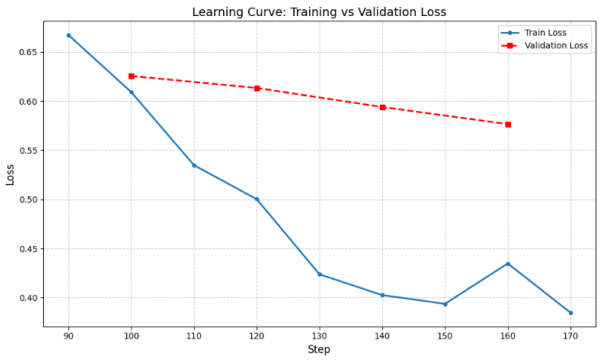


(b) Learning Curve (1000 ảnh)

Hình 2: Kết quả huấn luyện mô hình với tập dữ liệu 1000 ảnh. Hình bên trái là log quá trình train, hình bên phải là biểu đồ hội tụ.

	Step	Epoch	Train Loss	Val Loss	LR
8	90	1.59	0.6673	NaN	0.000136
9	100	1.76	0.6092	0.625581	0.000119
10	110	1.94	0.5348	NaN	0.000102
11	120	2.11	0.5002	0.613437	0.000086
12	130	2.28	0.4237	NaN	0.000069
13	140	2.46	0.4026	0.594019	0.000053
14	150	2.64	0.3937	NaN	0.000036
15	160	2.82	0.4348	0.576631	0.000020
16	170	3.00	0.3847	NaN	0.000003
17	171	3.00	NaN	NaN	NaN

(a) Training Log (2000 ảnh)



(b) Learning Curve (2000 ảnh)

Hình 3: Kết quả huấn luyện mô hình với tập dữ liệu 2000 ảnh. So sánh cho thấy sự thay đổi về tốc độ hội tụ.

Đánh giá quá trình huấn luyện với tập dữ liệu 1000 ảnh (Baseline) Thông số cuối cùng (tại Epoch 3): Train Loss: 0.5586 | Val Loss: 0.7152.

- **Sự ổn định của hàm mất mát (Loss Stability):** Hàm mất mát trên tập Train giảm đều đặn từ 2.24 xuống 0.55. Đường cong hội tụ mượt mà, không xuất hiện hiện tượng dao động mạnh (spikes) hay phân kỳ, chứng tỏ Learning Rate (khởi tạo $2e^{-4}$) và Batch Size hiệu dụng là phù hợp.
- **Khả năng tổng quát hóa:** Validation Loss giảm song hành với Train Loss qua các mốc đánh giá (từ 1.24 xuống 0.71). Khoảng cách giữa Train Loss và Val Loss (Generalization Gap) ở cuối chu kỳ là khoảng 0.15. Đây là mức chênh lệch nhỏ, cho thấy mô hình không bị Overfitting (quá khớp).
- **Hiệu quả của chiến lược Learning Rate:** Giai đoạn Warm-up (Step 0-50) giúp Loss giảm nhanh chóng. Giai đoạn Decay sau đó (Step 50-87) giúp mô hình tinh chỉnh trọng số để đạt điểm cực tiểu tốt hơn (Fine-grained).

Đánh giá quá trình huấn luyện với tập dữ liệu 2000 ảnh Thông số cuối cùng (tại Epoch 3): Train Loss: 0.3847 | Val Loss: 0.5766.

- **Hiệu suất vượt trội:** Với lượng dữ liệu gấp đôi, mô hình đạt mức Loss thấp hơn đáng kể. Train Loss giảm xuống còn 0.38 và Val Loss đạt 0.57.
- **Tiến trình hội tụ:** Số bước huấn luyện (Steps) tăng từ 87 lên 171 bước cho cùng 3 Epochs. Việc có nhiều lần cập nhật trọng số (gradient updates) hơn giúp mô hình học được kỹ hơn các đặc trưng phức tạp của dữ liệu.
- **Độ tin cậy:** Mặc dù Loss xuống thấp hơn, khoảng cách giữa Train và Val vẫn ổn định (~ 0.19). Validation Loss chưa có dấu hiệu đi ngang (plateau), gợi ý rằng mô hình vẫn có tiềm năng cải thiện nếu tăng thêm Epoch.

So sánh và nhận xét

Bảng dưới đây tóm tắt sự cải thiện hiệu suất khi tăng kích thước tập dữ liệu:

Bảng 2: So sánh hiệu quả huấn luyện giữa tập dữ liệu 1000 ảnh và 2000 ảnh

Tiêu chí	**Train 1000 ảnh**	**Train 2000 ảnh**	**Mức độ cải thiện**
Final Train Loss	0.5586	**0.3847**	Giảm $\sim 31\%$
Final Val Loss	0.7152	**0.5766**	Giảm $\sim 19\%$
Total Steps	87	171	-

Kết luận: Việc mở rộng tập dữ liệu từ 1000 lên 2000 ảnh đã mang lại hiệu quả rõ rệt:

- **Mật độ bao phủ không gian đặc trưng (Feature Space Coverage):** Với 1000 ảnh, mô hình có thể gặp phải tình trạng *Under-representation* (thiếu dữ liệu đại diện) ở một số lớp hoặc góc độ khó. Việc tăng lên 2000 ảnh giúp lấp đầy các khoảng trống trong không gian đặc trưng, giúp đường biên quyết định (decision boundary) của mô hình trở nên mượt mà và chính xác hơn, dẫn đến Validation Loss giảm sâu (0.57 vs 0.71).
- **Tác động của số bước cập nhật (Gradient Updates):** Mặc dù cả hai kịch bản đều chạy trong 3 Epochs, nhưng số lần mô hình được "học" (cập nhật trọng số) là khác nhau hoàn toàn.
  - Train 1000 ảnh: ≈ 87 bước cập nhật.
  - Train 2000 ảnh: ≈ 171 bước cập nhật (gấp đôi).

  Việc có nhiều bước cập nhật hơn (trong khi cùng một chiến lược Learning Rate Decay) cho phép thuật toán tối ưu hóa (Optimizer) dò tìm được điểm cực tiểu sâu hơn (deeper local minima) của hàm mất mát.
- **Giảm thiểu phương sai (Variance Reduction):** Theo định lý giới hạn trung tâm, khi kích thước mẫu (N) tăng lên, phương sai của ước lượng sẽ giảm đi. Trong trường hợp này, tập dữ liệu lớn hơn đóng vai trò như một bộ lọc nhiễu (noise filter), giúp mô hình tập trung học các đặc điểm tổng quát (general patterns) thay vì học vẹt các nhiễu cục bộ của tập dữ liệu nhỏ. Điều này giải thích tại sao đường Loss của 2000 ảnh ổn định hơn và ít dao động hơn.

4.3 Quy trình Suy luận và Hậu xử lí

Sau quá trình huấn luyện, mô hình được chuyển sang chế độ tối ưu cho suy luận (`FastVisionModel.for_inference`). Quy trình kiểm thử được thiết lập chặt chẽ để đảm bảo tính công bằng và chính xác.

1. Cấu hình sinh văn bản (Generation Config) Mỗi ảnh đầu vào được xử lý kèm theo câu lệnh (prompt) tiêu chuẩn: `<image> Free OCR.` với các tham số ràng buộc:

- **Max New Tokens:** 300. (Phù hợp với độ dài trung bình của các đoạn văn bản trong tập dữ liệu, ngăn chặn việc sinh quá dài không cần thiết).
- **Token ID:** Thiết lập chuẩn `pad_token_id` và `eos_token_id` từ tokenizer để đảm bảo mô hình nhận biết điểm dừng câu chính xác.

2. Hậu xử lý và Lọc nhiễu (Post-processing) Một vấn đề phổ biến của các mô hình VLM là lỗi "lặp từ vô hạn" (repetition loop). Để khắc phục, hệ thống áp dụng cơ chế lọc ngưỡng cứng:

- **Ngưỡng ký tự (Threshold):** 1500 ký tự.
- **Logic xử lý:** Nếu kết quả đầu ra vượt quá ngưỡng này (trong khi giới hạn token chỉ là 300), hệ thống xác định đây là lỗi lặp vô nghĩa và tự động loại bỏ kết quả (gán rỗng) để tránh làm sai lệch chỉ số đánh giá chung.
- **Làm sạch:** Loại bỏ các token kỹ thuật (như `<|endoftext|>`, `tensor(...)`) để thu được văn bản thuần túy.

5 Kết quả và đánh giá

5.1 Chiến lược dữ liệu (Data Strategy)

a) Lấy mẫu phân tầng (Stratified Sampling)

Do sự mất cân bằng trong bộ dữ liệu (các ảnh mức word chiếm đa số), em áp dụng chiến lược lấy mẫu phân tầng (30% Word, 30% Line, 40% Paragraph) để cân bằng giữa học ký tự và học ngữ cảnh.

Quy trình thực hiện:

1. Trích xuất dữ liệu theo tỷ lệ mục tiêu thành **pool** dữ liệu chung.
2. Trộn ngẫu nhiên (Shuffle) toàn bộ bể dữ liệu.
3. Chia tách Train (90%) và Validation (10%) từ bể này để đảm bảo sự tương đồng về phân phối thống kê.

b) Thiết kế kịch bản thực nghiệm

Hai kịch bản được thực hiện trên cùng một tập Test độc lập (100 mẫu) để so sánh:

- **Kịch bản 1 (1000 mẫu):** 900 Train / 100 Val.
- **Kịch bản 2 (2000 mẫu):** 1800 Train / 200 Val.

5.2 Chỉ số đánh giá

- **CER (Character Error Rate):** Đo lỗi cấp ký tự. Quan trọng vì tiếng Việt có hệ thống dấu phức tạp, sai dấu làm đổi nghĩa.
- **WER (Word Error Rate):** Đo lỗi cấp từ. Đánh giá khả năng ghép vần và hiểu ngữ cảnh.

5.3 Kết quả thực nghiệm

a) Kết quả định lượng (Quantitative Results)

Bảng 3 trình bày hiệu năng của các mô hình.

Mô hình	**CER**	**Δ CER**	**WER**	**Δ WER**
Base Model	1.4032	-	1.1309	-
Finetune (1000 mẫu)	0.2943	↓ 79.03%	0.5069	↓ 55.17%
Finetune (2000 mẫu)	**0.2844**	**↓ 79.73%**	**0.4750**	**↓ 58.00%**

Bảng 3: So sánh hiệu năng thực nghiệm trên tập Test (91 mẫu hợp lệ)

Nhận xét:

- **Phân tích hiện tượng chỉ số lỗi vượt quá 1.0 trên Base Model:** Quan sát Bảng 2, Base Model có chỉ số lỗi rất cao, cụ thể CER đạt 1.4032 và WER đạt 1.1309. Điều này là hợp lý về mặt toán học và phản ánh đúng bản chất của mô hình ngôn ngữ lớn (LLM) khi chưa được tinh chỉnh (Zero-shot).

  $$CER/WER = \frac{S + D + I}{N}$$

  Trong đó:

  - N: Tổng số ký tự/từ trong nhãn gốc (Ground Truth).
  - S (Substitutions): Số phép thay thế.
  - D (Deletions): Số phép xóa (bỏ sót).
  - I (Insertions): Số phép chèn (thêm vào).

  Do chưa được huấn luyện để dừng đúng lúc, Base Model thường xuyên gặp lỗi sinh lặp lại các cụm từ vô nghĩa hoặc mô tả dông dài thay vì chỉ trích xuất văn bản. Việc sinh ra số lượng ký tự dự đoán ($Prediction_Len$) lớn hơn nhiều so với nhãn gốc ($Groundtruth_Len$) khiến số lượng phép chèn ($Insertions$) tăng đột biến, đẩy tử số trong công thức Levenshtein vượt quá mẫu số.

- **Hiệu quả Finetuning:** LoRA giúp giảm lỗi đột biến (80% CER) so với Base model.
- **Tác động dữ liệu:** Tăng dữ liệu từ 1000 lên 2000 mẫu giúp WER giảm thêm 3%. Điều này chứng tỏ dữ liệu lớn giúp mô hình ghép từ và hiểu ngữ cảnh tốt hơn, dù khả năng nhận dạng ký tự (CER) đã bão hòa.

b) Phân tích lỗi định tính (Qualitative Analysis)

Bảng 4 minh họa sự khác biệt trong kết quả dự đoán thực tế.

ID	Ground Truth	Base Model Prediction	Finetune 87 bước Model (1000 ảnh) Prediction
0	cho giáo dục - đào tạo.	cho giào duc - đào tạo. *(Sai dấu câu)*	cho giáo dục - đào tạo. *(Chính xác)*
7	các	\[\sqrt{ab} \] *(Ảo giác toán học)*	các *(Chính xác)*
14	Thế nhưng những hiệp sĩ...	... the best, the best, the best, ... *(Lặp từ & Sai ngôn ngữ)*	Thế nhưng những luồng núi... *(Đúng tiếng Việt)*
66	như làm chạnh lòng tôi...	when some changes in... *(Ảo giác tiếng Anh)*	như lưới chà băng... *(Đúng tiếng Việt)*

Bảng 4: So sánh mẫu dự đoán giữa Base Model và Finetuned Model 87 bước

Phân tích các lỗi đặc trưng giữa Base Model và Finetune Model sau 87 bước (1000 ảnh):

- **Sai dấu câu (Diacritic Errors):** Base model thường bỏ hoặc sai dấu thanh (ví dụ: *giáo* → *giào*). Finetuned model đã học được quy luật chính tả tiếng Việt nên khắc phục tốt.
- **Lặp từ vô tận (Repetition Loop):** Với văn bản dài (ID 14), Base model bị lỗi lặp từ khóa (loop) do không xác định được điểm dừng. Finetuned model sinh ra câu hoàn chỉnh, gãy gọn.
- **Ảo giác ngôn ngữ (Hallucination):** Khi gặp chữ khó đọc, Base model thường "bịa" ra tiếng Anh hoặc công thức Toán (ID 7, 66). Ngược lại, Finetuned model tuân thủ nghiêm ngặt ràng buộc miền ngôn ngữ Tiếng Việt.

ID	Ground Truth	Finetune 87 bước Model (1000 ảnh)	Finetune 171 bước Model (2000 ảnh)
0	cho giáo dục - đào tạo.	cho giáo dục - đào tạo. *(Chính xác)*	cho giáo dục - đào tạo. *(Duy trì độ chính xác)*
14	Thế nhưng những hiệp sĩ...	Thế nhưng những **luồng núi**... *(Nhìn nhầm nét chữ)*	Thế nhưng những **hiệp sĩ**... *(Đã sửa đúng hoàn toàn)*
66	như làm chạnh lòng tôi...	như **lưới chà băng**... *(Câu vô nghĩa)*	như làm **chạnh lòng** tôi... *(Hiểu đúng ngữ cảnh)*
82	Hạnh phúc không nằm ở...	Hạnh phúc không **năm** ở... *(Thiếu dấu thanh)*	Hạnh phúc không **nằm** ở... *(Chính xác dấu câu)*

Bảng 5: So sánh kết quả dự đoán giữa Model được huấn luyện trên 1000 ảnh và 2000 ảnh

Phân tích các lỗi đặc trưng giữa 2 Finetune Model sau 1000 ảnh vs 2000 ảnh:

- **Lỗi nhận diện hình thái (Visual Ambiguity):** Với các nét chữ viết dài hoặc nối nét phức tạp, Model 1000 ảnh thường dự đoán dựa trên hình dáng bên ngoài dẫn đến sai lệch hoàn toàn (ví dụ: *hiệp sĩ* → *luồng núi*). Ngược lại, Model 2000 ảnh nhờ dữ liệu đa dạng hơn đã học được cách tách nét chính xác.

- **Sai lệch ngữ nghĩa (Semantic Validity):** Model 1000 ảnh đôi khi sinh ra các cụm từ có vẻ đúng chính tả nhưng vô nghĩa trong ngữ cảnh (ví dụ: *lưới chà băng*). Model 2000 ảnh cho thấy khả năng "hiểu" ngữ cảnh tốt hơn, đảm bảo câu sinh ra không chỉ đúng chữ mà còn hợp lý về nghĩa.
- **Độ nhạy với dấu thanh (Diacritic Sensitivity):** Mặc dù Model 1000 ảnh đã tốt hơn Base Model, nhưng vẫn còn sót lỗi dấu thanh ở các từ hiếm gặp hoặc bị mờ. Model 2000 ảnh (với Val Loss 0.57) thể hiện sự vượt trội trong việc bắt đúng các chi tiết nhỏ này, giảm thiểu tối đa tỉ lệ sai sót ký tự.

Xem thêm kết quả dự đoán tại: **Google Drive - Kết quả evaluation**

6 Kết luận

6.1 Những phát hiện chính (Key Findings)

Từ quá trình thực nghiệm và phân tích lỗi, đồ án rút ra 3 kết luận quan trọng về việc ứng dụng VLM cho bài toán OCR chữ viết tay tiếng Việt:

1. **LoRA biến đổi hành vi mô hình hiệu quả hơn là chỉ học tri thức:** Sự sụt giảm ngoạn mục của CER (từ 140% xuống ∼29%) không chỉ đến từ việc mô hình nhận dạng mặt chữ tốt hơn, mà chủ yếu do LoRA đã **căn chỉnh lại (re-align)** hành vi của mô hình. Từ trạng thái "tự do sáng tạo" (hallucination, lặp từ, chèn tiếng Anh) của Base Model, mô hình Fine-tune đã học được cách tuân thủ ràng buộc chặt chẽ của tác vụ OCR: chỉ trích xuất văn bản hiện hữu và dừng đúng lúc.
2. **Ngưỡng bão hòa của nhận dạng ký tự (CER Saturation):** Khi tăng dữ liệu từ 1000 lên 2000 mẫu, chỉ số CER gần như đi ngang (giảm rất nhẹ từ 0.29 xuống 0.28). Điều này cho thấy với kiến trúc hiện tại, khả năng nhận dạng hình thái ký tự (visual features) đã đạt giới hạn. Để giảm CER sâu hơn, cần cải thiện chất lượng ảnh đầu vào (pre-processing) hoặc thay đổi kiến trúc Encoder thay vì chỉ tăng số lượng mẫu đơn thuần.
3. **Dữ liệu lớn cải thiện tư duy ngữ nghĩa (Semantic Understanding):** Tuy CER bão hòa, nhưng WER vẫn tiếp tục giảm (cải thiện thêm 3% khi tăng dữ liệu). Điều này chứng minh rằng lượng dữ liệu lớn hơn giúp mô hình học được xác suất ngôn ngữ (Language Modeling) tốt hơn. Nó giúp mô hình tự sửa lỗi chính tả dựa trên ngữ cảnh (ví dụ: đoán đúng từ "hiệp sĩ" thay vì "luồng núi" nếu câu trước đó liên quan đến lịch sử), ngay cả khi nhìn hình ảnh không rõ.

6.2 Kết luận

Đồ án đã xây dựng thành công hệ thống OCR chữ viết tay tiếng Việt dựa trên kiến trúc Vision-Language Model hiện đại, đạt được các mục tiêu đề ra:

- **Tính khả thi trên tài nguyên hạn chế:** Chứng minh được hiệu quả của phương pháp QLoRA, cho phép huấn luyện mô hình 7B-8B tham số trên GPU đơn (Tesla T4 16GB) mà vẫn đạt độ chính xác cao (WER $\sim$ 0.47).

- **Khắc phục đặc thù tiếng Việt:** Mô hình giải quyết tốt các vấn đề nan giải của các thư viện OCR truyền thống (như Tesseract, EasyOCR) đối với tiếng Việt, đặc biệt là nhận diện dấu thanh và xử lý chữ viết tay tự do đa phong cách.
- **Hạn chế tồn đọng:** Mặc dù đã loại bỏ được lỗi lặp vô tận, mô hình đôi khi vẫn gặp lỗi ngữ nghĩa (nhìn nhầm từ có hình dáng giống nhau) do hạn chế về độ phân giải ảnh đầu vào.

Hướng phát triển tiếp theo: Tập trung vào việc tích hợp module sửa lỗi chính tả hậu xử lý (Post-processing LLM Correction) và mở rộng tập dữ liệu với các mẫu chữ viết tay có độ nghiêng/nhiễu cao hơn để tăng tính bền vững (robustness).

7 Tài liệu

[1] Tham khảo hướng dẫn finetune mô hình theo Unsloth: DeepSeek-OCR: How to Run Fine-tune.

[2] Tham khảo Vision Encoder - SigLIP: X. Zhai, et al., "Sigmoid Loss for Language Image Pre-Training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11975-11986, 2023.

[3] Tham khảo mô hình nền tảng (Base Model) DeepSeek-VL: H. Lu, et al., "DeepSeek-VL: Towards Real-World Vision-Language Understanding," *arXiv preprint arXiv:2403.05525*, 2024.

[4] Tham khảo phương pháp QLoRa: T. Dettmers, et al., "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.

8 Source code

[1] Source Notebook chuẩn bị dữ liệu: **Kaggle - [NLP] Prepare Input**, version 6

[2] Source Notebook finetune và đánh giá model: **Kaggle - [NLP] Deepseek OCR Finetune**, version 8 (train 1000 ảnh), version 9 (train 2000 ảnh)

9 Acknowledgement

Em xin chân thành cảm ơn các Thầy **Nguyễn Hồng Bửu Long, Đinh Điền, Lương An Vinh** đã tận tình hướng dẫn trong suốt quá trình học tập, cung cấp cho em những kiến thức nền tảng vững chắc để em có thể hoàn thành đồ án này.
Ngoài những kiến thức đã học và các tài liệu tham khảo được liệt kê, trong quá trình thực hiện, em có sử dụng công cụ AI Gemini như một trợ lý hỗ trợ giải thích thêm về các tham số finetune và viết bào cáo LaTeX.